به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین چهارم

مدرس: دکتر محمد قدسی، مهدی صفرنژاد — انتشار: ۷ اردیبهشت ۹۹

درهم‌سازی و مرتب‌سازی سریع

زمان تحویل این تمرین ۲۵ اردیبهشت ۱۴۰۰ می‌باشد. سؤالات را با دقت بخوانید و روی همه آن‌ها وقت بگذارید. تمرین‌های تئوری تحویل گرفته نمی‌شوند اما از آن‌ها سؤالات کوییز مشخص می‌شود. بنابراین روی سؤالات به خوبی فکر کنید و در کلاس‌های حل تمرین مربوطه شرکت کنید.

سؤال ۱. یک آرایه بیتی (صفر و یکی) m تایی فضای بسیار کمتری از آرایه‌ای از m پوینتر می‌گیرد. در صورتی که عناصری که داریم، داده اضافی همراه خود نداشته باشند و صرفا مقادیری بین 0 تا $m-1$ داشته باشند، نشان دهید چگونه روش آدرس‌دهی مستقیم را می‌توان با آرایه بیتی پیاده سازی کرد. پاسخ: این کار به شکل بسیار ساده قابل انجام است. فرض کنید یک آرایه بیتی 20 تایی داریم و مقادیر $2, 4, 6, 10, 15$ در آن قرار گرفته باشند. این را به راحتی با مقدار زیر می‌توان نمایش داد:

00101010001000010000

بر این اساس، اگر بدانیم فضای حالت ما به اندازه کافی کوچک است، روش آدرس‌دهی مستقیم از طریق این توابع قابل پیاده سازی خواهد بود؛ توجه کنید که V همان آرایه بیتی ما است:

---

procedure $Bitmap_Search(V, k)$

    if $V[k] \neq 0$ **then return** k
    return Null

procedure $Bitmap_Insert(V, k)$

    $V[k] = 1$

procedure $Bitmap_Delete(V, k)$

    $V[k] = 0$

سؤال ۲. قصد داریم که روش آدرس‌دهی مستقیم را بر روی یک آرایه بزرگ انجام بدهیم. در هنگام شروع کار مقادیر نامعتبری بر روی حافظه گرفته شده برای آرایه قرار دارند و به دلیل حجم بالای آن، مقداردهی اولیه آنان برای ما به صرفه نیست. روشی برای پیاده‌سازی آدرس‌دهی مستقیم روی این آرایه بزرگ ارائه دهید. هر عضو درون آرایه، باید فضایی از $O(1)$ بگیرد و عملیات‌های درج، حذف و جست‌وجو هم در $O(1)$ قابل انجام باشند. همچنین توجه کنید که ساختن حالت اولیه داده‌ساختار هم باید از $O(1)$ باشد و حق مقداردهی اولیه خانه‌های آرایه را نداریم. صرفا با یک درخواست می‌توانیم حافظه مدنظر را برای اعمال بعدیمان رزرو کنیم و ممکن است پیش از شروع کار مقادیر نامعتبری روی آرایه باشند.

راهنمایی: از یک آرایه (یا پشته) کمکی استفاده کنید که بفهمید مقدار درون هر خانه آرایه بزرگ معتبر است یا نه.

پاسخ:

داده ساختار کمکی ما یک پشته به نام S خواهد بود.

در ابتدا یک پشته خالی می‌سازیم و فضای مورد نیاز برای آرایه بزرگ را هم رزرو می‌کنیم اما کاری با مقداردهی اولیه آن نخواهیم داشت تا عملیات ما از $O(1)$ باشد. روش کار ما به این صورت خواهد بود که هر عضو در آرایه بزرگ در خود مقدار کلید و همچنین یک پوینتر به خانه از S را نگاه می‌دارد. هر عضو در S هم دقیقا به نقطه مشخص شده‌اش در آرایه بزرگ پوینتر دارد. بدین ترتیب از طریق تناظر درست پوینترها متوجه معتبر بودن هر خانه آرایه بزرگ خواهیم شد.

برای درج مقدار x یک پوینتر به نام y به S پوش می‌کنیم که آدرس خانه x ام آرایه بزرگ را دارد. در آرایه بزرگ و در خانه x ام هم علاوه بر کلید مد نظر، پوینتر به خانه نظیر در S یعنی y قرار می‌دهیم.

برای جست‌وجوی وجود مقدار x به خانه x ام آرایه بزرگ می‌رویم. اگر این خانه پوینتری به نقطه‌ای از S داشت که آن نقطه در S هم پوینتری به خانه x ام آرایه بزرگ داشت، یعنی این خانه معتبر است و واقعا مقداری در آن درج شده است. در غیر این صورت یعنی مقداری در آن درج نشده و داده نامعتبری از قبل در آن قرار گرفته است.

برای حذف، نگاه می‌کنیم که خانه x ام آرایه بزرگ به چه مقداری در S اشاره می‌کند. ابتدا آدرس این خانه را در $temp$ ذخیره می‌کنیم. سپس بالایی ترین عنصر پشته را که مثلا به خانه z ام آرایه بزرگ اشاره می‌کند از پشته پاپ کرده و مقدار درون آن را در خانه‌ای که آدرسش را در $temp$ ذخیره کرده بودیم قرار می‌دهیم تا پشته ما حفره نداشته باشد. حال به خانه z ام آرایه بزرگ رفته و مقدار اشاره‌گر مربوط به پشته را در آن به $temp$ تغییر می‌دهیم. همه این عملیات ها در $O(1)$ قابل انجام خواهند بود.

توجه: پیاده سازی چنین سیستمی با زبان‌هایی که مستقیما از پوینتر و اشاره‌گر پشتیبانی نمی‌کنند کمی دشوار خواهد بود. برای همین اگر خواستید این سیستم را به شکل عملی پیاده‌سازی کنید توصیه می‌شود از C یا C++ استفاده کنید.

سؤال ۳. شیوه خاصی از تابع درهم‌سازی به روش تقسیم را در نظر بگیرید که در آن $h(k) = k \bmod m$ که در آن $m = 2^p - 1$ و k هم یک رشته از کاراکترهاست که بیانگر عددی در مبنای 2^p است. نشان دهید که اگر دو رشته x و y داشته باشیم که با جابه‌جا کردن کاراکترهای یکی از آن‌ها بتوانیم به دیگری برسیم (یکی از آن‌ها جایگشتی از دیگری باشد)، هر دوی این رشته‌ها با تابع درهم‌سازی ذکر شده به یک خانه نگاشت می‌شوند. مثالی از یک کاربرد واقعی بیاورید که وجود چنین خاصیتی نامطلوب باشد.
پاسخ: نشان می‌دهیم که هش هر رشته در روش گفته شده برابر جمع ارقامش به پیمانه $2^p - 1$ خواهد بود. برای این کار از استقرا استفاده می‌کنیم. برای پایه استقرا یک رشته تک کاراکتری را در نظر بگیرید. در این صورت مقدار آن به پیمانه $m = 2^p - 1$ محاسبه می‌شود. پس برای پایه استقرا حکم برقرار است. حال برای گام استقرا فرض کنید رشته $w = w_1 w_2$ باشد که از n کاراکتر تشکیل شده است و w_1 شامل $n - 1$ کاراکتر پرارزش تر باشد و w_2 هم کاراکتر آخر آن باشد. در این صورت

$$h(w) = h(w_1)2^p + h(w_2) \bmod 2^p - 1$$

$$h(w) = h(w_1) + h(w_2)$$

این یعنی که برای یک رشته با طول دلخواه، حاصل تابع درهم ساز معادل حاصل جمع هش تمامی آن به جز رقم آخر و هش رقم آخر به صورت جداگانه است. این موضوع نشان می‌دهد که هش هر رقم به طور جداگانه تاثیر دارد و در نتیجه اگر یک عدد k رقمی به

فرم $x = x_1x_2\cdots x_k$ داشته باشیم

$$h(x) = h(x_k) + h(x_1\cdots x_{k-1}) = h(x_k) + h(x_{k-1}) + h(x_1\cdots x_{k-2}) = \cdots = h(x_1) + \cdots h(x_k)$$

خواهد بود.

برای مثال حالتی که چنین سیستمی مشکل ساز می‌شود، فرض کنید که یک دیکشنری انگلیسی طراحی می‌کنید که لغات براساس این تابع در آن ذخیره می‌شوند و کاراکترهای مورد استفاده ما هم کدهای ASCII باشند و در نتیجه مبنا $2^8 = 256$ و $m = 255$ باشد. در این صورت مثلا کلماتی نظیر STOP,TOPS,SPOT,POTS که همگی معنی دار هم هستند به یک slot نگاشت خواهند شد.

سؤال ۴. فرض کنید که یک جدول درهم‌سازی با n جایگاه [1] داریم. برخوردها هم از طریق روش زنجیره‌سازی حل می‌شوند. فرض کنید که در ابتدا n کلید در جدول درج شده اند و احتمال نگاشت هر کلید به هر کدام از جایگاه‌ها هم برابر است. فرض کنید که M حداکثر تعداد کلیدهایی باشد که در یک جایگاه قرار گرفته‌اند. در این مسئله قصد داریم اثبات کنیم که حد بالای $E[M]$، یعنی حد بالای امیدریاضی M از $O(\log n/\log\log n)$ است.

آ. به شکل شهودی توجیه کنید که Q_k یعنی احتمال این که دقیقا k کلید در یک جایگاه مشخص قرار گرفته باشند به شکل زیر بدست می‌آید:

$$Q_k = (\frac{1}{n})^k(1-\frac{1}{n})^{n-k}\binom{n}{k}$$

ب. فرض کنید که P_k بیانگر احتمال این باشد که $M = k$. یعنی احتمال این که جایگاه با بیش‌ترین کلید دارای k کلید باشد را با P_k نمایش می‌دهیم. نشان دهید $P_k \le nQ_k$

با استفاده از تقریب استرلینگ، نشان دهید که $Q_k < e^k/k^k$. تقریب استرلینگ به صورت $n! = \sqrt{2\pi n}(\frac{n}{e})^n(1+\Theta(\frac{1}{n}))$ است.

پ. نشان‌دهید که یک ثابت $c > 1$ وجود دارد به طوری که به ازای $k_0 = c\log n/\log\log n$ داشته باشیم $Q_{k_0} < 1/n^3$

از این موضوع نتیجه بگیرید که به ازای $k \ge k_0$ داریم: $P_k < 1/n^2$

ت. از بخش‌های بالا نتیجه بگیرید که

$$E[M] \le \Pr\{M > \frac{c\log n}{\log\log n}\} \times n + \Pr\{M \le \frac{c\log n}{\log\log n}\} \times \frac{c\log n}{\log\log n}$$

و از این موضوع نتیجه بگیرید که $E[M] = O(\log n/\log\log n)$

پاسخ: برای قسمت اول، فرض کنید که Q_k احتمال قرار گرفتن دقیقا k کلید در یک جایگاه مشخص باشد. تعداد راه‌های انتخاب این k کلید به صورت $\binom{n}{k}$ است. آن‌ها با احتمال $\frac{1}{n}$ به همان جایگاه مشخص نگاشت می‌شوند و $n-k$ کلید باقی‌مانده هم هر یک با احتمال $1-\frac{1}{n}$ به یکی از $n-1$ جایگاه دیگر نگاشت می‌شوند. در نتیجه:

[1] Slot

$$Q_k = \left(\frac{1}{n}\right)^k \left(\frac{n-1}{n}\right)^{n-k} \binom{n}{k}$$

برای قسمت دوم، می‌دانیم که حد بالای احتمال این که جایگاه با بیش‌ترین تعداد کلید دقیقا k کلید داشته باشد، برابر وجود جایگاه‌ای با دقیقا k کلید است. n جایگاه وجود دارد که می‌توانند این k کلید را در خود جای دهند. در نتیجه $P_k \leq nQ_k$

برای قسمت سوم، با توجه به تقریب استرلینگ می‌توانیم نتیجه بگیریم که $\binom{n}{k} \leq (\frac{ne}{k})^k$ در نتیجه: $Q_k \leq \frac{(n-1)^{n-k}}{n^n} \left(\frac{ne}{k}\right)^k \leq e^k/k^k$

برای قسمت چهارم در اصل هدف م این است که به $Q_{k_0} < n^{-3}$ برسیم. از فرمول هر دو طرف log می گیریم و سعی می‌کنیم در لگاریتمشان این فرمول را ایجاد کنیم. با لگاریتم گرفتن داریم:

$$\frac{c \lg n}{\lg \lg n}(\lg e + \lg \lg \lg n - \lg c - \lg \lg n) < -3 \lg n$$

با تقسیم دو طرف بر $-\log \log n$ داریم

$$3 < -\frac{c \lg e}{\lg \lg n} - \frac{\lg \lg \lg n}{\lg \lg n} + \frac{\lg c}{\lg \lg n} + c$$

در $n \to \infty$ همه جملات صفر می‌شوند به جز جمله c. در نتیجه با انتخاب $c = 4$ خواسته سوال به ازای $n > N$ محقق خواهد شد. با توجه به نتیجه قسمت دوم، می‌توانیم براساس نزولی بودن Q_k برحسب k نتیجه بگیریم که به ازای $k \geq k_0$ مقدار $P_k < 1/n^2$ است.

برای قسمت آخر، می‌توانیم از طریق حالت بندی به عبارت مد نظر برسیم. در اصل می‌توانیم این طور در نظر بگیریم که برای M های بزرگتر از $c \lg n/\lg \lg n$ مقدار M را در حد بالا n در نظر گرفته و برای مقادیر کوچکتر از آن برابر خود $c \lg n/\lg \lg n$ در نظر بگیریم. بدین ترتیب یک حد بالا برای امیدریاضی M خواهیم داشت. چون امید ریاضی به صورت حاصلضرب احتمال در مقدار متغیر تصادفی است و ما در هر دو حالت حد بالایی را برای مقدار متغیر تصادفی در نظر گرفته ایم. با توجه به شرط قسمت قبلی می دانیم که $nP(M > \frac{c \lg n}{\lg \lg n}) < 1$ و همچنین می‌توان با توجه به ذات احتمالاتی نتیجه گرفت که $\sum_{i=1}^{d} P(M = i) \leq 1$ به ازای $d = c \lg n/\lg \lg n$ در نتیجه با جایگذاری به راحتی به این نتیجه می‌رسیم که $E[M] \in O(\lg n/\lg \lg n)$

سؤال ۵. تعداد n نفر در یک سالن حضور دارند. می‌خواهیم با انتخاب $(k-1)$ سرگروه از بین این افراد، کاری کنیم که اگر همگی آن‌ها به ترتیب قد در یک صف بایستند، سرگروه‌ها صف را به k دسته‌ی مساوی (اختلاف تعداد افراد دسته‌ها می‌تواند تا یکی دو نفر باشد) تقسیم کرده باشند. فرض کنید لیست نامرتبی از قد افراد در اختیار دارید.

الگوریتمی از مرتبه‌ی $O(nk)$ برای این کار ارائه دهید.

الگوریتمی از مرتبه‌ی $O(nlgk)$ برای این کار ارائه دهید.

پاسخ:

کافی است برای هر سرگروه یک بار اندیس جایگاه‌اش در آرایه‌ی مرتب شده را محاسبه کرده ($O(1)$) و الگوریتم SELECT (پیدا کردن k امین عنصر در $O(n)$) را صدا بزنیم.

از الگوریتم SELECT و تقسیم و حل استفاد ه می‌کنیم. ابتدا الگوریتم را برای $\lfloor \frac{k}{2} \rfloor$ صدا زده، از نتیجه‌ی به دست آمده به عنوان pivot استفاده می‌کنیم و آرایه را با آن partition بندی می‌کنیم. سپس به طور بازگشتی برای دو بخش چپ و راست، سرگروه‌ها پیدا می‌شوند. با اندکی تأمل به این نتیجه می‌رسید که عمق درخت بازگشت $\log k$ است و در نتیجه مرتبه‌ی زمانی الگوریتم $O(n \log k)$ می‌باشد.

سؤال ۶. در کلاس درس، با الگوریتم SELECT آشنا شدید که در مرتبه‌ی زمانی خطی، k امین کوچک‌ترین عنصر آرایه‌ای نامرتب را به دست می‌داد. در عملکرد این الگوریتم مشاهده کردید که عناصر آرایه به دسته‌های پنج‌تایی تقسیم می‌شوند. حال اگر در الگوریتمی مشابه، عناصر را به دسته‌های هفت‌تایی تقسیم کنیم و سایر بخش‌های الگوریتم مانند قبل باشد:

حداقل فاصله‌ی میانه‌ی میانه‌ها از دو سر آرایه چقدر خواهد بود؟

فرض کنید برای مرتب‌سازی دسته‌ها از مرتب‌سازی حبابی (Bubble sort) استفاده می‌شود. رابطه‌ی بازگشتی الگوریتم جدید را بنویسید و علت وجود هر جمله‌ی آن را به اختصار شرح دهید.

نشان دهید الگوریتم همچنان عملکرد زمانی خطی دارد.

اگر به جای دسته‌های پنج‌تایی یا هفت‌تایی از دسته‌های سه‌تایی استفاده شود چطور؟ مرتبه‌ی زمانی الگوریتم همچنان خطی خواهد بود؟

پاسخ:

عناصر میانی نصف دسته‌ها از میانه‌ی میانه‌ها کوچک‌تر هستند. پس:

$$\frac{1}{2} \times 4 \times \frac{n}{7} = \frac{2n}{7}$$

حداقل فاصله‌ی میانه‌ی میانه‌ها از دو سر آرایه $\frac{2n}{7}$ می‌باشد.

مراحل الگوریتم و مرتبه‌ی زمانی مورد نیاز هر مرحله به قرار زیر است:

(آ)دسته‌بندی آرایه‌ی اصلی به گروه‌های هفت‌تایی: O(n)

(ب)مرتب‌سازی هر دسته (در بدترین حالت نیاز به 7×6 مقایسه در هر دسته داریم) که نتیجه می‌دهد تعداد کل مقایسه‌ها:

$$\frac{n}{7} \times 7 \times 6 \times C = 6Cn = O(n)$$

(ج)تشکیل آرایه‌ای از عناصر میانی دسته‌ها: $O(n)$

(د)پیدا کردن بازگشتی میانه‌ی‌میانه‌ها: $T(\frac{n}{7})$

(ه)پارتیشن‌بندی آرایه‌ی اصلی با استفاده از میانه‌ی میانه‌ها به عنوان محور: $O(n)$

(و)پیداکردن k امین کوچک‌ترین عنصر به طور بازگشتی: $T(\frac{5n}{7})$
در نتیجه رابطه‌ی بازگشتی به صورت زیر نوشته می‌شود:

$$T(n) = T(\frac{n}{7}) + T(\frac{5n}{7}) + O(n)$$

حدس:

$$T(1) = c_1, O(n) = c_2 n \qquad T(n) \leq 10(c_1 + c_2)n$$
$$\rightarrow T(n) \leq (\frac{6}{7})(10(c_1 + c_2)) + c_2 n = (\frac{60}{7})c_1 n + (\frac{67}{7})c_2 n \leq 10(c_1 + c_2)n$$
$$\rightarrow T(n) = O(n)$$

در آن صورت:

$$T(n) = T(\lceil \frac{n}{3} \rceil) + T(\frac{4n}{6}) + O(n) \geqslant T(\frac{n}{3}) + T(\frac{2n}{3}) + O(n)$$

حدس:

$$T(m) = T(\frac{m}{3}) + T(\frac{2m}{3}) + O(m) \rightarrow T(m) \geqslant cm \log m$$

$$\rightarrow T(n) \geqslant c(\frac{n}{3}) \log (\frac{n}{3}) + c(\frac{2n}{3}) \log (\frac{2n}{3}) + O(n)$$
$$\geqslant cn \log n + O(n)$$

پس در این حالت مرتبه‌ی زمانی خطی ندارد.

سؤال ۷. الگوریتم مرتب‌سازی سریع را به گونه‌ای تغییر داده‌ایم که در هر مرحله، آرایه‌ی ورودی را با نسبت‌های α و $1 - \alpha$ تقسیم می‌کند و $0 < \alpha \leq \frac{1}{2}$. کم‌ترین و بیشترین عمق یک برگ در درخت بازگشتی را برحسب n و α محاسبه کنید .

پاسخ: برای یافتن کم‌ترین عمق همواره باید بخش کوچکتر (متناظر با α) انتخاب شود.

$$\alpha^h n = 1 \rightarrow h = \log_\alpha \frac{1}{n} = -\frac{\log n}{\log \alpha}$$

برای بیشترین عمق نیز:

$$(1 - \alpha)^h n = 1 \rightarrow h = \log_{1-\alpha} \frac{1}{n} = -\frac{\log n}{\log 1 - \alpha}$$

سؤال ۸. فرض کنید که مجموعه کلیدهای ما به صورت بردارهای $[x_1, x_2, ..., x_k]$ باشند، با این فرض که هر کدام از x_iها میتوانند از بازه ی $\{0, 1, ..., N - 1\}$ انتخاب شده باشند که N یک مقدار اول است. نشان دهید که خانواده زیر برای این تابع یک

خانواده جهانی است:

$$H = \{h|h(x) = (r_1x_1 + r_2x_2 + ... + r_kx_k)\%N\}$$

که در آن تمام r_i ها از بازه $\{0, 1, ..., N-1\}$ انتخاب شده‌اند. پاسخ: اگر H یک خانواده از توابع درهم سازی باشد، در این صورت H یک خانواده جهانی است اگر:

$$\forall x, y \in U, x \neq y \qquad \Pr_{h\in H}[h(x) = h(y)] \leq \frac{1}{N}$$

که در آن N اندازه آرایه است. برای محاسبه ی احتمال هم به روش زیر عمل میکنیم:

$$\frac{favourableevents}{totalevents} = \Pr$$

که صورت همان حالت های تساوی $h(x)$ و $h(y)$ است. تعداد حالت های کل، تمام توابع ممکن در مجموعه هستند.

با توجه به تعریف مجموعه، r_i میتواند از بازه ی ۰ تا $N-1$ انتخاب شود. چون تعداد r ها k تا است و هر کدام N مقدار مختلف میتوانند بگیرند، پس N^k تا تابع داریم که همان مخرج کسر هستند.

برای محاسبه‌ی تعداد حالت های پیشامد دو کلید مختلف $x = [x_1, x_2, ..., x_k]$ و $y = [y_1, y_2, ..., y_k]$ را در نظر میگیریم. چون متفاوت هستند پس حداقل یکی از مولفه هایشان با هم متفاوت هستند. مثلا فرض کنید مولفه امz این دو با هم متفاوت هستند.

حالا میخوهیم ببینیم چه تعداد حالت هایی وجود دارد که h(x) برابر با h(y) باشد.

$$\begin{aligned}
&h(x) = h(y)\\
&\textstyle\sum_{i=1}^{k} r_ix_i \overset{N}{\equiv} \sum_{i=1}^{k} r_iy_i\\
&\textstyle\sum_{i=1}^{k} r_ix_i - \sum_{i=1}^{k} r_iy_i \overset{N}{\equiv} 0\\
&\textstyle\sum_{i=1}^{k} r_i(x_i - y_i) \overset{N}{\equiv} 0\\
&\textstyle r_j(x_j - y_j) + \sum_{i=1,i\neq j}^{k} r_i(x_i - y_i) \overset{N}{\equiv} 0\\
&\textstyle r_j(x_j - y_j) \overset{N}{\equiv} -\sum_{i=1,i\neq j}^{k} r_i(x_i - y_i)\\
&\textstyle r_j \overset{N}{\equiv} -\sum_{i=1,i\neq j}^{k} r_i(x_i - y_i)(x_j - y_j)^{-1}
\end{aligned}$$

حالا باید تعداد حالتهای مختلف را محاسبه کنیم. r_i دارای N حالت مختلف است. r_j هم به صورت یکتا توسط عبارت سمت راست تعیین میشود. این عبارت $k-1$ جمله دارد. پس به طور کلی N^{k-1} حالت مختلف وجود دارد و:

$$Pr = \frac{N^{k-1}}{N^k}$$

پس حکم برقرار است.

سؤال ۹. در جدول درهم سازی با استفاده از روش وارسی خطی، تابع درهم سازی برای جدولی با اندازه هشت به صورت زیر است:

key	A	B	C	D	E	F	G	H
hash	۲	۶	۲	۴	۴	۵	۴	۱

اگر جدول درهم سازی در ابتدا تهی باشد، به چند حالت میتوان این عناصر را در جدول درج کرد تا در نهایت جدول زیر تولید شود؟

i	A	۱	۲	۳	۴	۵	۶	۷
T(i)	G	H	A	C	D	E	B	F

پاسخ:

$Hash(A) = 2, Hash(C) = 2$
$T(2) = A, T(3) = C$

بنابراین به دو مورد زیر C بعد از A هش میشود.

$Hash(D) = 4, Hash(E) = 4$
$T(4) = D, T(5) = E$

و E بعد از D هش میشود.

$Hash(E) = 4, Hash(B) = 6, Hash(F) = 5$
$T(5) = E, T(6) = B, T(7) = F$

و F بعد از E و B هش میشود.

$Hash(D) = 4, Hash(E) = 4, Hash(G) = 4$
$Hash(B) = 6, Hash(F) = 5$
$T(5) = E, T(6) = B$
$T(7) = F, T(0) = G$

و G باید بعد از F و E و B هش شود.

حالا تعداد جایشگت های مختلف را محاسبه میکنیم. H هیچ محدودیتی نداشته است پس ۸ جایگشت مختلف دارد. A و C هم با توجه به یکی از محدودیت ها دارای $C(7, 2)$ یعنی ۲۱ جایگشت مختلف هستند. با توجه به شروط دو و سه و چهار هم متوجه میشویم که ترتیب هش شدن به صورت D و E و F و G است و B هم قبل از F باید هش شود پس ۳ حالت دارد.

برابراین به طور کلی $8 * 3 * 21 = 504$ حالت مختلف برای ساختن این آرایه داریم.

موفق باشید